Deleting Rows with DELETE

Lecture Notes · Game DB Junior

In this lesson

- How to delete rows with **DELETE FROM** and **WHERE**
- Why the WHERE statement must be **precise**
- How to put a deleted row back with **INSERT INTO**

1. The DELETE syntax

Deleting a row from a table has a **very simple syntax**:

```
DELETE FROM cities
WHERE name = 'Tokyo';
```

Part	What it does
DELETE FROM	Keywords: we want to delete rows from a table
cities	The name of the table we want to delete a row from
WHERE name = 'Tokyo'	Says exactly which record or records we want to delete

Postgres looks at all the rows in cities and finds any row with the name Tokyo. Here that is the first row, and it gets deleted.

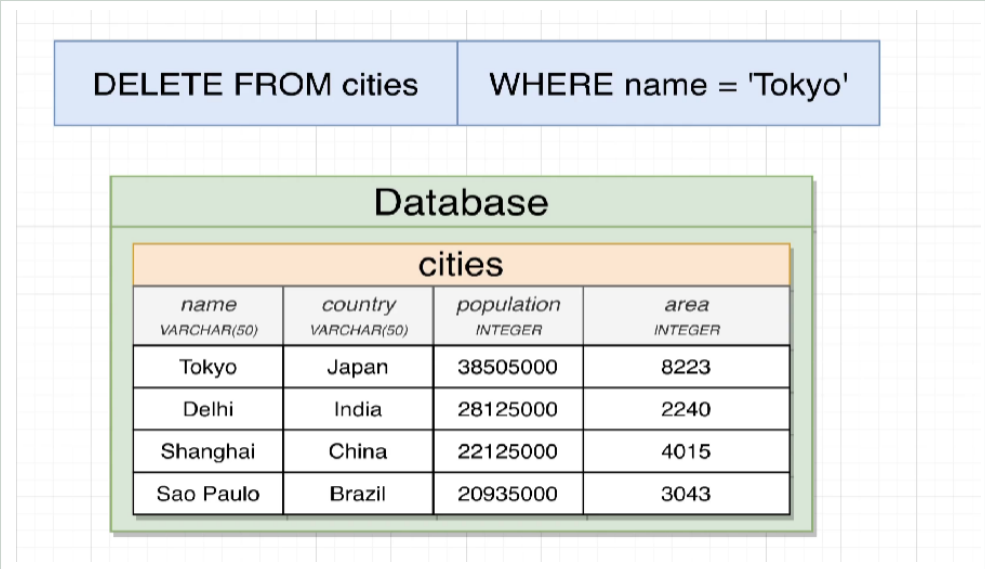


Figure 1 — DELETE FROM cities WHERE name = 'Tokyo' deletes the Tokyo row

2. Be careful with WHERE

WHERE works the same way as in UPDATE. If your WHERE statement matches **many rows**, you will delete **many records**.

```
DELETE FROM cities
WHERE name <> 'Tokyo';
```

- name <> 'Tokyo' (name is not equal to Tokyo) finds Delhi, Shanghai and Sao Paulo.
- All **three rows** are deleted in one go.

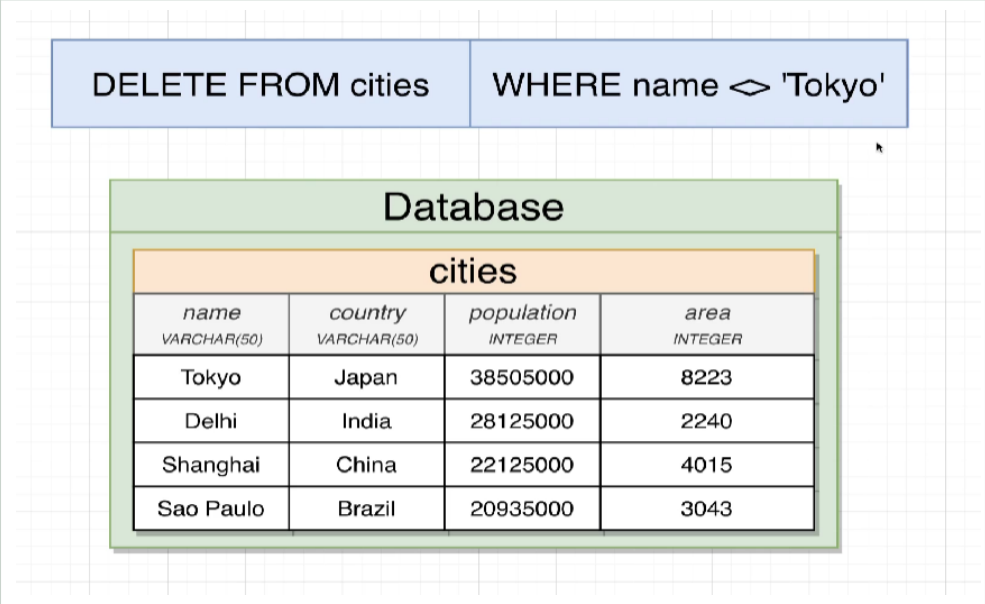


Figure 2 — DELETE FROM cities WHERE name <> 'Tokyo' deletes three rows

Remember: just like UPDATE, be very careful and precise with the WHERE statement. Make sure you always select exactly the row you want to delete. Later in the course we will see some tools to always point to exactly the right row.

3. Example: delete Tokyo

- We run **DELETE FROM cities WHERE name = 'Tokyo';**
- Then we fetch all the cities again with **SELECT * FROM cities;** (we can just click the play button to rerun the query).
- Tokyo is **no longer** in the list of cities.

4. Putting Tokyo back

We want to add Tokyo back in, in case we run more queries. To insert a record we use **INSERT INTO**, the table name, the column names, then **VALUES** with the values for the new row:

```
INSERT INTO cities (name, country, population, area)
VALUES ('Tokyo', 'Japan', 38505000, 8223);
```

The insert is successful. When we select the cities table again, we see Tokyo listed there.

5. Running the code (my practice)

Step 1: delete the row where the name is Tokyo. The Messages tab shows **DELETE 1**: one row was deleted.

```sql
-- Delete rows from the "cities" table
DELETE FROM cities
-- Only delete the row where the city name is 'Tokyo'
WHERE name = 'Tokyo';
```

Figure 3 — DELETE with comments

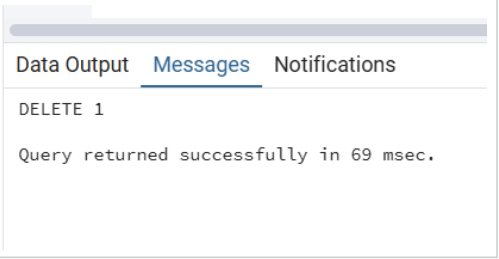
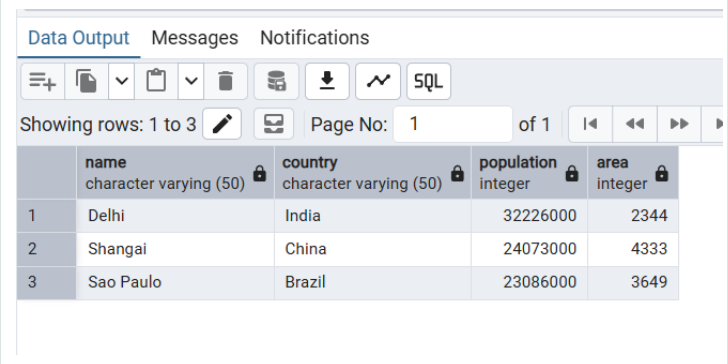


Figure 4 — DELETE 1

Step 2: SELECT * FROM cities. Only 3 rows are left. Tokyo is gone.



	name character varying (50)	country character varying (50)	population integer	area integer
1	Delhi	India	32226000	2344
2	Shangai	China	24073000	4333
3	Sao Paulo	Brazil	23086000	3649

Figure 5 — Delhi, Shangai, Sao Paulo

Step 3: insert Tokyo back. I used the values from my table: 39505000 and 8231. The Messages tab shows **INSERT 0 1**: one row was added.

```
-- Insert the record back.
INSERT INTO cities (name, country, population, area)
VALUES
    ('Tokyo', 'Japan', 39505000, 8231);
```

Figure 6 — INSERT INTO to put Tokyo back

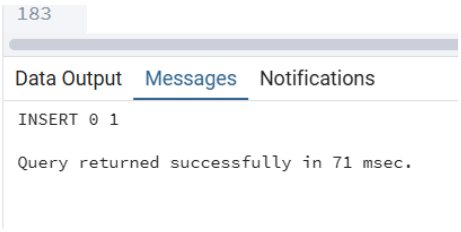
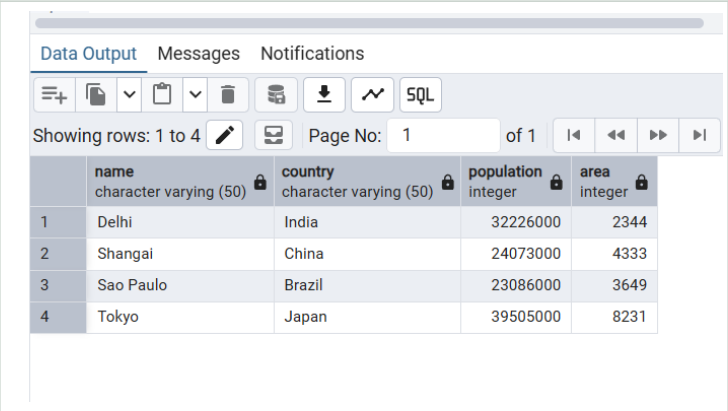


Figure 7 — INSERT 0 1

Step 4: SELECT * FROM cities again. Tokyo is back, and we have 4 rows.



	name character varying (50)	country character varying (50)	population integer	area integer
1	Delhi	India	32226000	2344
2	Shangai	China	24073000	4333
3	Sao Paulo	Brazil	23086000	3649
4	Tokyo	Japan	39505000	8231

Figure 8 — Tokyo is back in the table

6. Summary

Idea	Remember
DELETE FROM table	Choose the table we want to delete rows from
WHERE ...	Choose exactly which rows to delete
Many matches	If WHERE matches many rows, all of them are deleted in one go
Be precise	Just like UPDATE, write the WHERE statement very carefully
Check the result	Run SELECT * FROM cities to see the change

Next lesson: a quick exercise or two on updates and deletes.

Sources
Lecture transcript and lecture images (Figures 1–2)
My own practice screenshots (Figures 3–8)